

Assignment - II

I Classical Linear Search

1a Algorithm (with Mathematical Notation)

• **Input:** a list $L = [a_0, a_1, \dots, a_{N-1}]$ of integers and a target $t \in \mathbb{Z}$.

• **Steps:**

1. Initialize $i \leftarrow 0$

2. While $i < N$:

• Compare a_i with t .

• If $a_i = t$, output the index i and stop.

• Otherwise, set $i \leftarrow i + 1$ and continue.

3. If the loop finishes without finding t , output "Not Found"

4. END

• The algorithm outputs an index i iff there exists $i \in \{0, \dots, N-1\}$ with $a_i = t$; else it reports "Not Found".

• Complexity: Time $O(N)$, space $O(1)$.

• Mathematical Representation:

$$\text{LinearSearch}(A, t) = \begin{cases} i & \text{if } a_i = t \text{ for some } i \in \{1-N\} \\ \text{"NOT FOUND"} & \text{if otherwise} \end{cases}$$

1b def linearSearch(arr, t):

for i in range(len(arr)):

if arr[i] == t:

return i

return -1

arr = [3, 5, 2, 9, 1, 7, 4]

try:

t = int(input("Enter target integer: "))

except ValueError:

print("Invalid Input")

raise SystemExit(1)

idx = linear_search(arr, t)

if idx != -1:

print(f"Found at index {idx}")

else:

print("Not Found")

2. Quantum Search - Grover's Algorithm

Grover's algorithm searches unsorted database faster than classical search.

- Classical search complexity: $O(N)$

- Quantum search complexity: $O(\sqrt{N})$

2. Grover's Algorithm works on Quantum states for 2 qubits, the possible states are:

$|00\rangle, |01\rangle, |10\rangle, |11\rangle$

This algorithm finds one target state faster using amplitude amplification.

- Notation:

→ Uniform superposition $|s\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle$

→ Oracle $O: O|x\rangle = (-1)^{f(x)} |x\rangle$ (phase flip on the marked item)

→ Diffusion $D = 2|s\rangle\langle s| - I$ (Inversion about the mean)

- Steps:

1. Initialization into equal superposition

→ Start from $|00\dots 0\rangle$ and apply Hadamard H on each qubit so all basis states have equal amplitude.

2. Oracle operation to mark the correct element
→ apply a selective phase flip to the basis state $|x^*\rangle$ that encodes the target
3. Diffusion Operator (amplitude amplification)
→ Reflect the amplitudes about their average to increase the marked state's amplitude.
→ For 2 qubits: $H \otimes H \rightarrow X \otimes X \rightarrow CZ \rightarrow X \otimes X \rightarrow H \otimes H$
4. Measurement of the final state
→ Measure in the computational basis; the marked item appears with high probability after $\approx \frac{\pi}{4} \sqrt{N}$ iterations

- Two-qubit example ($N=4$): target $|11\rangle$
→ Oracle: CZ applies a -1 phase to $|11\rangle$
→ Diffusion: $H \otimes H, X \otimes X, CZ, X \otimes X, H \otimes H$

2b. Oracle Program

- Mathematical Representation

$$|s\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle, \quad O = I - 2|x^*\rangle\langle x^*|, \quad \theta = \frac{\pi}{4}$$

$$D = 2|s\rangle\langle s| - I, \quad G = DO, \quad \sin\theta = \frac{1}{\sqrt{N}}$$

$$G^k |s\rangle = \sin((2k+1)\theta) |x^*\rangle + \cos((2k+1)\theta) |s\rangle, \quad k_{opt} \approx \left\lceil \frac{\pi}{4\theta} \right\rceil \approx \left\lceil \frac{\pi}{4} \sqrt{N} \right\rceil, \quad \text{queries } O(\sqrt{N}), \quad N=4 \Rightarrow \theta = \frac{\pi}{6}, \quad k_{opt} = 1$$

2b. Qiskit Program

from math import sqrt, pi, floor
from qiskit import QuantumCircuit, transpile

def k_opt_from_n(n):

$$N = 2^{**n}$$

return math.ceil(int(floor((pi/4) * sqrt(N))))

def apply_oracle_two_qubits(qc, target):

if target == "11":

qc.c2(0, 1)

return

if target == "00":

qc.x([0, 1])

qc.c2(0, 1)

qc.x([0, 1])

return

if target == "01":

qc.x(0)

qc.c2(0, 1)

qc.x(0)

return

if target == "10":

qc.x(1)

qc.c2(0, 1)

qc.x(1)

return

raise ValueError("target must be one of '00', '01', '10', '11'")

def apply_diffusion_two_qubits(qc):

qc.h([0, 1])

qc.x([0, 1])

qc.c2(0, 1)

qc.x([0, 1])

qc.h([0, 1])

Manik Gupta
290952

```
def grover-two-qubits(target="11", iterations=None):  
    qc = QuantumCircuit(2,2)  
    qc.h([0,1])  
    if iterations is None:  
        iterations = k-opt-from-n()  
    for _ in range(iterations):  
        apply-oracle-two-qubits(qc, target)  
        apply-diffusion-two-qubits(qc)  
    qc.measure([0,1], [0,1])  
    return qc
```

```
qc = grover-two-qubits(target="11")  
try:  
    from qiskit_aer import AerSimulator  
    sim = AerSimulator()  
    compiled = transpile(qc, sim)  
    result = sim.run(compiled, shots=1024).result()  
    print(result.get_counts())  
except Exception:  
    print(qc.draw())
```